Flash Read and Write Experiment

1. Flash Filesystem Overview

A **filesystem partition** (FAT/LittleFS by default) is allocated in the internal Flash of the ESP32-S3. MicroPython can operate on files through the standard `open()` / `read()` / `write()` interfaces, and the data is **retained after power-off**. This experiment implements interactive Flash data management: append, read, delete, and clear.

Typical application scenarios:

Application	Example
Configuration storage	Wi-Fi SSID/password, device parameters
Data logging	Historical sensor data, operation logs
Calibration data	ADC offset, calibration coefficients
Offline cache	Temporarily store data when the network is down

2. Core Concepts

2.1 MicroPython Filesystem

MicroPython automatically mounts the Flash filesystem at startup. Under the root directory `/`, the standard Python file API (`open` / `read` / `write`) can be used for reading and writing.

2.2 Data Storage Format

In this experiment, the first line stores the **total record count**, and each subsequent line stores one user data entry. On the first run the file does not exist; the code automatically creates it and writes `0` on the first line. After the first append, the first line is updated to the actual record count:

```
0                   ← First created, count is 0
Hello ESP32-S3      ← 1st entry (the first line is updated to 1 after the
append)
MicroPython Flash   ← 2nd entry
...
```

Each operation first reads all lines into a list, then writes everything back after adding/deleting — suitable for small data volumes (< tens of KB).

2.3 Core API

```
# Open file
f = open(f_path, "w")    # Write mode (overwrite)
f = open(f_path, "r")    # Read mode
f = open(f_path, "a")    # Append mode

# Read & write
content = f.read()              # Read all content
lines = content.splitlines()   # Split into lines
f.write("hello\n")              # Write one line
f.close()                       # Close (must!)
```

---

3. Hardware Dependencies

This experiment only uses the internal Flash of the ESP32-S3, **no external hardware required**.

3.1 ESP32-S3 Hardware Features

Feature	Description
Total Flash capacity	Usually 8MB or 16MB (query with `esp32.flash_size()`)
Filesystem partition	The factory MicroPython firmware mounts the remaining Flash space as FAT/LittleFS at path `/`
Filesystem capacity	Total Flash minus firmware/partition-table usage, usually several MB (query with `os.statvfs('/')`)
Write endurance	NOR Flash typically endures 100,000+ erase/write cycles; suitable for configuration/log scenarios, not for high-frequency writes every second
Power-loss protection	`open/close` syncs to Flash immediately; data is not lost on power-off. However, sudden power loss during writing may corrupt the file (FAT has no journal), so it is recommended to verify critical data after writing

---

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  ├─ Initialize: check whether the file exists; if not, create it and write the
first-line count "0\n"
  ├─ Print the menu: [1 Append / 2 Read / 3 Delete / 4 Clear]
  |
  └─ while True:
       ├─ input()  wait for REPL command input
       |
       ├─ "1" Append:
       |    ├─ Read all lines → parse count (first line)
       |    ├─ count++ → append a new line
       |    └─ Write everything back to the file
       |
       ├─ "2" Read:
       |    ├─ Read all lines → skip count=0 or empty
       |    └─ Print line by line
       |
       ├─ "3" Delete a single entry:
       |    ├─ Enter the index → delete that line
       |    ├─ count-1, update the first line
       |    └─ Write back
       |
       └─ "4" Clear:
            └─ Write "0\n" (reset)
```

---

5. Source Code Details

Complete source code: `5.Source Collection → MicroPython → 3.Serial Communication and Storage System → 4. FLASH → flash_fs.py`

5.1 File Initialization

```
# Create the file if it does not exist
try:
    f = open(f_path, "r")
    f.close()
except:
    f = open(f_path, "w")
    f.write("0\n")       # First line = count
    f.close()
```

> `try/except` is a reliable way to check whether a file exists — some MicroPython versions do not support `os.path.exists()`.

5.2 Append Data (Command 1)

```
# Read the current content
f = open(f_path, "r")
lines = f.read().splitlines()
f.close()

# Parse and update the counter
count = 0
if len(lines) > 0:
    count = int(lines[0])
count += 1
lines[0] = str(count)

# Append a new line
lines.append(input("Content: "))

# Write all lines back
f = open(f_path, "w")
for s in lines:
    f.write(s + "\n")
f.close()
print("Saved.")
```

5.3 Read All (Command 2)

```
f = open(f_path, "r")
lines = f.read().splitlines()
f.close()

count = int(lines[0]) if len(lines) > 0 else 0
if count == 0 or len(lines) <= 1:
    print("No data.")
else:
    print("Flash data:")
    for i in range(1, len(lines)):
        print("%d: %s" % (i, lines[i]))
```

5.4 Delete a Single Entry (Command 3)

```
num = int(input("Index: "))
f = open(f_path, "r")
lines = f.read().splitlines()
f.close()

if num < 1 or num >= len(lines):
    print("Invalid index.")
else:
    del lines[num]                  # Delete the line
    lines[0] = str(len(lines) - 1)  # Update the count
    with open(f_path, "w") as f:
        for s in lines:
            f.write(s + "\n")
    print("Deleted.")
```

5.5 Clear (Command 4)

```
f = open(f_path, "w")
f.write("0\n")
f.close()
print("Cleared.")
```

---

6. Operation Procedure

6.1 Flashing and Running

Thonny IDE operation steps: See `MicroPython → 1.Development Basics → 3.Thonny IDE`.

1. Open `flash_fs.py` with Thonny IDE and click Run
2. Enter commands in the **REPL/Shell area** to interact (number 1~4 + Enter)

6.2 Expected Behavior

After startup, the program prints the menu and waits for user input. Enter the corresponding number to perform append/read/delete/clear operations. The data is saved in the Flash filesystem and remains after a soft reset or power-off.

Flash space is usually several MB; the first-line count method in this experiment is suitable for **hundreds of entries** of small data. For large amounts of data storage, it is recommended to write line by line in append mode.

6.3 Serial Output Example

```
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
1
Content: Temperature 25.5C
Saved.
1
Content: Humidity 60%
Saved.
```

```
1
Content: ESP-IDF
Saved.
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
3
Index: 1
Deleted.
2
Flash data:
1: Humidity 60%
2: ESP-IDF
4
Cleared.
2
No data.
```

```
MPY: soft reboot
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
1
Content: Temperature 25.5C
Saved.
1
Content: Humidity 60%
Saved.
1
Content: ESP-IDF
Saved.
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
3
Index: 1
Deleted.
2
Flash data:
1: Humidity 60%
2: ESP-IDF
4
Cleared.
2
No data.
```

Stored data before power-off:

```
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
1
Content: Temperature 25.5C
Saved.
1
Content: Humidity 60%
Saved.
1
```

```
Content: ESP-IDF
Saved.
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
```

```
MPY: soft reboot
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
1
Content: Temperature 25.5C
Saved.
1
Content: Humidity 60%
Saved.
1
Content: ESP-IDF
Saved.
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
```

Read after power-off and restart (data not lost):

```
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
```

```
===== Internal Flash Test =====
1 Append 2 Read 3 Delete 4 Clear
2
Flash data:
1: Temperature 25.5C
2: Humidity 60%
3: ESP-IDF
```

6.4 Verification Method

Action	Expected result
Append 3 entries	Read back 3 entries, indices 1~3
Delete index 2	Entry #3 shifts to #2
Read after clear	Displays "No data."
Soft reset & read	Data persists (non-volatile)
Enter an invalid index	Shows "Invalid index."

7. Common Issues

Symptom	Cause	Solution
Data lost after soft reset	File not close()d	Always `f.close()` after each read/write
`OSError: [Errno 28] ENOSPC`	Insufficient Flash space	Delete useless files, or use `os.statvfs('/')` to check the remaining space
`ValueError: invalid literal`	The first-line count is corrupted	Manually delete the data file and run again
Chinese garbled text	Encoding issue	MicroPython defaults to UTF-8; the Thonny IDE REPL also needs to be set to UTF-8